



**TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A
DISTANCIA**

PROGRAMACION 3

Trabajo Practico Integrador - FoodStore

Alumno: Leonel Jesus Aballay

Comision: 6

Índice

1. Introducción
2. Objetivos
3. Descripción General del Proyecto
4. Tecnologías Utilizadas
5. Arquitectura del Sistema
6. Modelo de Datos
7. Implementación del Sistema
8. Persistencia y Gestión de Transacciones
9. Conclusiones
10. Bibliografía

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Integrador correspondiente a la materia **Programación III**, cuyo objetivo fue diseñar e implementar un sistema de gestión comercial aplicando los principales conceptos abordados durante la cursada.

El proyecto fue desarrollado utilizando una arquitectura dividida en dos componentes principales: un **Frontend**, encargado de la interacción con el usuario, y un **Backend**, responsable de la lógica de negocio, la persistencia de datos y la administración de la base de datos.

El sistema permite administrar categorías, productos, usuarios y pedidos, incorporando además un conjunto de reportes que facilitan la consulta de la información almacenada.

Durante el desarrollo se aplicaron tecnologías ampliamente utilizadas en el ámbito profesional, tales como **Java**, **JPA**, **Hibernate**, **H2 Database**, **HTML5**, **CSS3**, **JavaScript** y **TypeScript**, implementando una arquitectura modular que favorece el mantenimiento, la reutilización del código y futuras ampliaciones del sistema.

Asimismo, se incorporaron conceptos fundamentales como programación orientada a objetos, persistencia de datos, relaciones entre entidades, transacciones, consultas JPQL y el patrón Repository, permitiendo construir una aplicación robusta y organizada.

2. Objetivos

Objetivo General

Desarrollar un sistema de gestión comercial que permita administrar categorías, productos, usuarios y pedidos, aplicando una arquitectura organizada que integre un frontend desarrollado con tecnologías web y un backend implementado en Java utilizando JPA e Hibernate para la persistencia de la información.

Objetivos Específicos

- Implementar una interfaz gráfica desarrollada con HTML5, CSS3, JavaScript y TypeScript.
- Desarrollar un backend en Java aplicando los principios de la programación orientada a objetos.
- Implementar persistencia de datos utilizando JPA y Hibernate.
- Gestionar la información mediante una base de datos H2.
- Aplicar el patrón Repository para organizar el acceso a los datos.
- Implementar operaciones CRUD para las principales entidades del sistema.

- Utilizar baja lógica para preservar la integridad de la información.
- Implementar consultas mediante JPQL.
- Garantizar la consistencia de los datos utilizando transacciones.
- Generar reportes que permitan consultar información relevante del sistema.

3. Descripción General del Proyecto

El sistema desarrollado corresponde a una aplicación de gestión comercial cuyo propósito es facilitar la administración de la información relacionada con productos y pedidos de un negocio.

La aplicación fue diseñada utilizando una arquitectura cliente-servidor, separando claramente la interfaz de usuario de la lógica de negocio y de la persistencia de datos.

El **Frontend** proporciona una interfaz amigable que permite al usuario interactuar con las distintas funcionalidades del sistema, mientras que el **Backend** administra las reglas de negocio, las validaciones y la persistencia de la información.

Dentro del sistema se implementaron cinco módulos principales:

Gestión de Categorías

Permite registrar, modificar, consultar y realizar la baja lógica de las categorías utilizadas para organizar el catálogo de productos.

Gestión de Productos

Administra toda la información correspondiente a los productos comercializados, incluyendo nombre, descripción, precio, stock disponible, estado de disponibilidad e imagen asociada.

Cada producto pertenece a una categoría determinada, permitiendo mantener correctamente organizado el catálogo.

Gestión de Usuarios

Permite registrar y administrar la información correspondiente a los clientes del sistema, almacenando sus datos personales, correo electrónico, contraseña, teléfono y rol.

Además, incorpora búsquedas mediante correo electrónico para facilitar su localización.

Gestión de Pedidos

Representa el módulo principal del sistema.

Permite generar pedidos seleccionando un usuario y agregando distintos productos, verificando automáticamente el stock disponible, calculando los subtotales, el importe total del pedido y actualizando las existencias de cada producto.

Toda esta operación se ejecuta dentro de una única transacción para garantizar la integridad de la información almacenada.+

Reportes

El sistema incorpora distintos reportes que permiten consultar la información registrada.

Entre ellos se incluyen:

- Productos por categoría.
- Pedidos por usuario.
- Pedidos por estado.
- Total facturado.

Estos reportes fueron implementados utilizando consultas JPQL sobre la base de datos H2.

Introducción a la Arquitectura

Con el objetivo de facilitar el mantenimiento del proyecto y respetar las buenas prácticas de desarrollo de software, el sistema fue organizado mediante una arquitectura en capas.

Cada componente cumple una función específica dentro de la aplicación, permitiendo separar la presentación, la lógica de negocio y la persistencia de los datos.

Esta organización mejora considerablemente la legibilidad del código y permite que futuras modificaciones puedan realizarse sin afectar el funcionamiento general del sistema.

4. Tecnologías Utilizadas

Para el desarrollo del Trabajo Práctico Integrador se emplearon distintas tecnologías tanto para el frontend como para el backend, seleccionadas por su robustez, facilidad de integración y por los conocimientos adquiridos durante la materia.

Frontend

El desarrollo de la interfaz de usuario se realizó utilizando tecnologías estándar para aplicaciones web.

HTML5 fue utilizado para definir la estructura de las páginas, organizando formularios, menús, tablas y los distintos componentes visuales del sistema.

CSS3 permitió diseñar la apariencia de la aplicación, estableciendo colores, tipografías, distribución de los elementos y una interfaz clara para el usuario.

JavaScript fue utilizado para incorporar comportamiento dinámico a la aplicación, permitiendo manipular el DOM, validar formularios y responder a las acciones realizadas por el usuario.

TypeScript se incorporó para mejorar la organización del código mediante tipado estático, facilitando la detección de errores durante el desarrollo y permitiendo una estructura más mantenible.

Durante el desarrollo del frontend también se utilizó **LocalStorage** para almacenar temporalmente la información de usuarios, sesiones y carrito de compras, simulando el funcionamiento de una base de datos antes de la integración con el backend.

Backend

El backend fue desarrollado utilizando **Java**, implementando una arquitectura orientada a objetos basada en entidades y repositorios.

Para la persistencia de datos se utilizó **JPA (Java Persistence API)** junto con **Hibernate**, permitiendo mapear automáticamente las entidades Java sobre las tablas de la base de datos.

La utilización de **Lombok** permitió reducir considerablemente la cantidad de código repetitivo mediante la generación automática de constructores, métodos de acceso y patrones Builder.

Como sistema gestor de base de datos se empleó **H2 Database**, una base de datos relacional liviana que facilitó el desarrollo, las pruebas y la persistencia local de la información.

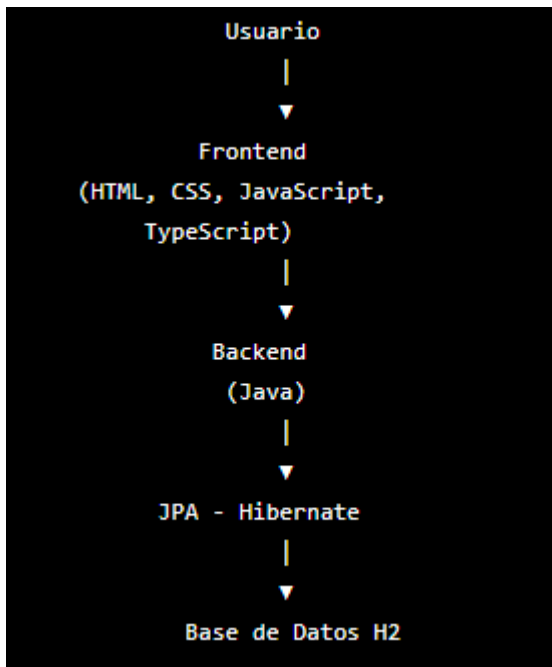
Finalmente, **Gradle** fue utilizado como herramienta de construcción del proyecto y administración de dependencias.

5. Arquitectura del Sistema

Con el objetivo de mantener una aplicación organizada y escalable, el proyecto fue desarrollado siguiendo una arquitectura en capas.

Cada una de ellas posee una responsabilidad específica, evitando el acoplamiento entre la interfaz de usuario, la lógica de negocio y la persistencia de los datos.

La arquitectura general del sistema puede representarse mediante el siguiente esquema:



El usuario interactúa inicialmente con el frontend, donde se presentan las distintas opciones del sistema.

Las solicitudes son procesadas posteriormente por el backend, encargado de aplicar las reglas de negocio y realizar las validaciones correspondientes.

Finalmente, mediante JPA y Hibernate, la información es almacenada o recuperada desde la base de datos H2.

Esta organización facilita el mantenimiento del proyecto y permite que cada componente pueda evolucionar de manera independiente.

6. Modelo de Datos

El modelo de datos fue diseñado utilizando entidades Java que representan los principales objetos del sistema.

Cada entidad fue mapeada a una tabla de la base de datos mediante anotaciones de JPA, estableciendo además las relaciones necesarias entre ellas.

Las entidades implementadas fueron las siguientes:

Entidad	Función
Categoría	Clasifica los productos del sistema.

Entidad	Función
Producto	Almacena la información de cada producto, incluyendo precio, stock y disponibilidad.
Usuario	Representa a los clientes registrados en el sistema.
Pedido	Registra las compras realizadas por los usuarios.
DetallePedido	Almacena cada uno de los productos incluidos en un pedido.

Todas las entidades heredan de una clase base denominada **Base**, la cual proporciona el identificador único y el atributo utilizado para la baja lógica.

Relaciones entre Entidades

Las relaciones implementadas mediante JPA permiten representar correctamente el funcionamiento del negocio.

- Una **Categoría** puede contener múltiples **Productos**.
- Un **Usuario** puede realizar varios **Pedidos**.
- Un **Pedido** puede contener múltiples **Detalles de Pedido**.
- Cada **Detalle de Pedido** hace referencia a un único **Producto**.

Estas relaciones fueron implementadas utilizando las anotaciones @OneToMany y @ManyToOne, permitiendo que Hibernate administre automáticamente la persistencia de los objetos relacionados.

Durante la implementación del módulo de pedidos se utilizó además CascadeType.ALL, logrando que los detalles asociados se almacenen automáticamente al persistir un pedido, simplificando el código y manteniendo la consistencia de la información.

Organización del Backend

Para facilitar el mantenimiento del proyecto, el backend fue organizado en distintos paquetes según la responsabilidad de cada componente.

La estructura principal del proyecto es la siguiente:

```
com.tu.programacion3
|
├─ entities
├─ repository
├─ enums
├─ interfaces
├─ utils
├─ temp
└─ Main.java
```

Esta organización permitió mantener una separación clara entre las entidades del sistema, el acceso a datos, las enumeraciones, las interfaces y las clases auxiliares, favoreciendo un código más limpio y fácil de mantener.

7. Implementación del Sistema

Una vez definido el modelo de datos y la arquitectura de la aplicación, se procedió al desarrollo de los distintos módulos que conforman el sistema. Cada uno fue implementado de manera independiente, respetando una estructura organizada basada en el patrón Repository y utilizando JPA para la persistencia de la información.

Todas las funcionalidades fueron desarrolladas siguiendo un mismo flujo de trabajo: validación de los datos ingresados, ejecución de la lógica de negocio y persistencia mediante el repositorio correspondiente.

7.1 Gestión de Categorías

El módulo de Categorías permite administrar la clasificación de los productos registrados dentro del sistema.

Las operaciones implementadas fueron:

- Alta de categorías.
- Modificación de categorías existentes.
- Baja lógica.

- Consulta de categorías activas.

Durante la carga de una nueva categoría el sistema solicita el nombre y la descripción correspondiente, almacenando posteriormente la información en la base de datos mediante el repositorio específico.

La baja lógica fue implementada marcando el registro como eliminado, evitando su eliminación física y preservando la integridad de la información.

7.2 Gestión de Productos

El módulo de Productos administra toda la información relacionada con el catálogo del comercio.

Cada producto almacena:

- Nombre.
- Descripción.
- Precio.
- Stock.
- Estado de disponibilidad.
- Imagen.
- Categoría asociada.

Las funcionalidades desarrolladas incluyen:

- Alta.
- Modificación.
- Baja lógica.
- Consulta de productos.
- Búsqueda por categoría.

Durante la carga y modificación se implementaron validaciones para controlar que los datos ingresados fueran correctos y que el producto permaneciera asociado a una categoría existente.

Además, el sistema controla permanentemente el stock disponible para garantizar el correcto funcionamiento del módulo de pedidos.

7.3 Gestión de Usuarios

El módulo de Usuarios permite registrar y administrar los clientes del sistema.

Cada usuario almacena información personal como:

- Nombre.
- Apellido.
- Correo electrónico.
- Contraseña.
- Teléfono.
- Rol.

Las operaciones implementadas fueron:

- Alta.
- Modificación.
- Baja lógica.
- Consulta de usuarios.
- Búsqueda mediante correo electrónico.

Durante el desarrollo se incorporó una validación que evita registrar usuarios duplicados utilizando el correo electrónico como identificador principal.

7.4 Gestión de Pedidos

El módulo de Pedidos constituye la funcionalidad principal del sistema, ya que integra la mayor parte de la lógica de negocio desarrollada.

Las operaciones implementadas fueron:

- Creación de pedidos.
- Modificación del estado.
- Baja lógica.
- Consulta de pedidos activos.
- Búsqueda por usuario.

- Búsqueda por estado.

Para crear un pedido, el sistema solicita inicialmente el usuario que realizará la compra y la forma de pago correspondiente.

Posteriormente permite incorporar uno o varios productos, verificando automáticamente que cada uno exista, se encuentre disponible y posea stock suficiente.

Una vez finalizada la selección de productos, el sistema calcula automáticamente el subtotal de cada detalle y el importe total del pedido.

Finalmente, se actualiza el stock de los productos involucrados y toda la información es almacenada en la base de datos mediante una única transacción.

7.5 Reportes

El sistema incorpora un módulo de reportes que permite consultar información relevante para la administración del negocio.

Los reportes implementados fueron:

- Productos por categoría.
- Pedidos realizados por un usuario.
- Pedidos filtrados por estado.
- Total facturado.

Estos reportes fueron desarrollados utilizando consultas JPQL sobre la base de datos, permitiendo obtener información de manera rápida y organizada.

7.6 Validaciones Implementadas

Durante el desarrollo del sistema se incorporaron distintas validaciones con el objetivo de garantizar la consistencia de la información almacenada.

Entre las principales pueden mencionarse:

- Verificación de existencia de categorías.
- Verificación de existencia de productos.
- Verificación de existencia de usuarios.
- Control de stock disponible.
- Validación de cantidades mayores a cero.

- Verificación de disponibilidad de productos.
- Control de estados de los pedidos.
- Validación de registros activos mediante baja lógica.

Estas verificaciones permiten reducir errores durante la utilización del sistema y mejorar la confiabilidad de la aplicación.

7.7 Integración entre Frontend y Backend

Aunque el frontend y el backend fueron desarrollados como proyectos independientes, ambos fueron diseñados siguiendo una estructura que facilita su integración.

El frontend fue desarrollado utilizando archivos JSON y LocalStorage para simular el almacenamiento de información durante las primeras etapas del proyecto.

Posteriormente, el backend incorporó persistencia mediante JPA y Hibernate utilizando una base de datos H2.

Esta organización permite que, en una futura evolución del sistema, el frontend pueda reemplazar la obtención de datos desde archivos locales por consultas realizadas a una API REST implementada en el backend, sin necesidad de modificar la lógica de presentación.

Conclusión de la Implementación

La implementación del sistema permitió desarrollar una aplicación completa de gestión comercial, integrando correctamente las funcionalidades del frontend con la lógica de negocio implementada en el backend.

La organización modular del proyecto, junto con el uso de JPA, Hibernate y el patrón Repository, permitió construir una solución ordenada, reutilizable y fácilmente mantenible.

Asimismo, las validaciones incorporadas y la correcta administración de la persistencia garantizaron la consistencia de la información almacenada y el correcto funcionamiento de todas las operaciones implementadas.

8. Persistencia y Gestión de Transacciones

Uno de los aspectos más importantes del desarrollo del backend fue la implementación de la persistencia de datos utilizando **JPA (Java Persistence API)** junto con **Hibernate** como proveedor de persistencia.

Para administrar las operaciones sobre la base de datos se utilizó el componente **EntityManager**, encargado de gestionar el ciclo de vida de las entidades y ejecutar las operaciones de inserción, modificación, consulta y actualización de los datos.

Con el objetivo de mantener el código organizado, todas las operaciones relacionadas con la persistencia fueron implementadas siguiendo el patrón **Repository**, centralizando el acceso a la base de datos en clases específicas para cada entidad.

Este enfoque permitió reutilizar código, simplificar el mantenimiento del proyecto y separar claramente la lógica de negocio del acceso a los datos.

Gestión de Transacciones

La operación más importante del sistema corresponde a la creación de un pedido, ya que involucra simultáneamente distintas entidades como Usuario, Pedido, Producto y DetallePedido.

Para garantizar la integridad de la información, toda esta operación fue implementada dentro de una única transacción.

Durante la creación del pedido el sistema realiza automáticamente las siguientes acciones:

- Recupera el usuario seleccionado.
- Obtiene los productos incorporados al pedido.
- Verifica la disponibilidad y el stock de cada producto.
- Genera los detalles correspondientes.
- Calcula el subtotal de cada producto.
- Calcula el importe total del pedido.
- Actualiza el stock disponible.
- Persiste el pedido junto con todos sus detalles.

Si todas estas operaciones finalizan correctamente, la transacción se confirma mediante el método **commit()**, almacenando definitivamente la información en la base de datos.

En caso de producirse cualquier error durante la ejecución, se ejecuta automáticamente **rollback()**, deshaciendo todos los cambios realizados y garantizando que la base de datos nunca quede en un estado inconsistente.

Esta implementación permitió asegurar que un pedido se registre completamente o que no se registre en absoluto, manteniendo la integridad de la información almacenada.

Consultas JPQL

Para la recuperación de información se utilizaron consultas **JPQL (Java Persistence Query Language)**.

A diferencia de SQL, JPQL trabaja directamente sobre las entidades Java y no sobre las tablas físicas de la base de datos.

Durante el desarrollo se implementaron consultas para:

- Buscar productos por categoría.
- Buscar usuarios mediante correo electrónico.
- Buscar pedidos por usuario.
- Buscar pedidos por estado.

Estas consultas fueron utilizadas tanto en las funcionalidades del sistema como en la generación de los distintos reportes implementados.

9. Conclusiones

El desarrollo de este Trabajo Práctico Integrador permitió aplicar de forma conjunta los principales contenidos abordados durante la materia Programación III, integrando conocimientos relacionados con programación orientada a objetos, persistencia de datos, desarrollo web y arquitectura de software.

Durante la implementación del proyecto se desarrolló un sistema completo de gestión comercial compuesto por un frontend y un backend, permitiendo administrar categorías, productos, usuarios y pedidos, además de generar distintos reportes sobre la información almacenada.

En el backend se aplicaron tecnologías ampliamente utilizadas en el desarrollo profesional, como **Java**, **JPA**, **Hibernate**, **Lombok** y **H2 Database**, implementando una arquitectura organizada basada en el patrón **Repository** y utilizando **EntityManager** para administrar la persistencia y las transacciones.

Por su parte, el frontend fue desarrollado utilizando **HTML5**, **CSS3**, **JavaScript** y **TypeScript**, construyendo una interfaz intuitiva y organizada que facilita la interacción del usuario con el sistema.

Asimismo, el proyecto permitió comprender la importancia de la separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y la persistencia de datos, favoreciendo la mantenibilidad y escalabilidad de la aplicación.

Finalmente, este trabajo representó una experiencia práctica muy valiosa, permitiendo integrar diferentes tecnologías en un único proyecto y consolidar los conocimientos adquiridos durante la cursada.

10. Bibliografía

- Oracle. *Java Documentation*. <https://docs.oracle.com/>
- Jakarta EE. *Jakarta Persistence API (JPA)*. <https://jakarta.ee/>
- Hibernate ORM. *Official Documentation*. <https://hibernate.org/orm/documentation/>
- H2 Database Engine. <https://www.h2database.com/>
- Project Lombok. <https://projectlombok.org/>
- Gradle Build Tool. <https://gradle.org/>
- Mozilla Developer Network (MDN). <https://developer.mozilla.org/>
- W3Schools. <https://www.w3schools.com/>